

V4 Working Notes

Initiative Sequence

1. **Configuration Flow Optimisation** – establish the precedence model across manifest, suite, test, and CLI layers so script context, execution parameters, and fixtures resolve deterministically.
2. **Execution Defaults & Discovery** – apply the resolved defaults inside `scripts/run-tests-gordon.js` and refactor `lib/tests.js` for target-aware, sorted discovery via `tests/<target>/<suite>/test_*.js`.
3. **Sequenced Test Chains** – extend metadata with `chain`, `order`, and `reset` hints, validate continuity, and record chain context plus reset mode in per-test artefacts.
4. **Diagnostics & Developer Tooling** – add verbose tracing and retry logic around CLI interactions, expose `--list-suites`, `--list-tests`, `--only`, and honour `--quiet`.
5. **Requirement Traceability** – let tests declare requirement identifiers, persist them via `writePerTestRecord`, and generate suite coverage reports.
6. **Documentation & Guides** – refresh authoring guides to cover the expanded metadata schema, sequencing rules, traceability, and configuration precedence examples for sync/async/hardware suites.

Initiative Detail

1. Configuration Flow Optimisation

- Classify configuration into `scriptContext`, `executionParams`, and `fixtures`; document supported keys per class.
- Define precedence: board manifest → suite-level `testConfig.json` → per-test metadata header → CLI overrides.
- Create `lib/config.js` to load/merge sources, validate schema, and emit the final config object consumed by runners.
- Store suite defaults alongside tests (`tests/<target>/<suite>/testConfig.json`) and require a matching `"suite"` tag for sanity checks.
- Persist the resolved config for each test at `results/<stamp>/<board>/runner-metadata/<testId>.json` (toggleable later).
- Warn on unknown keys or conflicting directives (e.g., incompatible reset modes) before executing tests.

2. Execution Defaults & Discovery

- Surface manifest-driven parameters (timeouts, resets, CLI config, baud) inside the Gordon runner before metadata overrides apply.
- Replace the hard-coded suite lists in `lib/tests.js` with globbed discovery per target; sort results deterministically.

3. Sequenced Test Chains

- Support `chain`, `order`, and `reset` metadata to orchestrate multi-step workflows, including tests that require or skip resets between steps.

- Validate that chains have contiguous steps; skip remaining steps automatically when a prerequisite fails and record the reason.

4. Diagnostics & Developer Tooling

- Add verbose tracing and optional retries around `preloadStorageFiles` / `sendViaCLI` to capture FIFO_FULL scenarios (payload sizes, timings, stderr).
- Implement CLI utilities for discovery (`--list-suites`, `--list-tests`) and filtering (`--only <pattern>`), while keeping noise controllable via `--quiet`.

5. Requirement Traceability

- Allow metadata to declare `requirements` arrays; carry them into per-test JSON artefacts and provide a report that maps requirements to suites/tests.

6. Documentation & Guides

- Update `docs/test-writing-guide.md` (and related references) with new metadata examples, chain usage, requirement tagging, and the configuration precedence model for synchronous, asynchronous, and hardware-focused suites.

Configuration Data Map

Script Context

- **Elements:** `fixturesInjection`, additional `contextInjection` snippets.
- **Sources:**
 - Suite `testConfig.json` (`scriptContext.fixtureVar`, `scriptContext.inject`).
 - Test metadata header (`contextInjection`, existing CLI config hooks).
 - CLI `--fixtures <path>` selecting JSON from `configs/`.

Execution Parameters

- **Existing:** `timeoutMs`, `preUploadDelayMs`, `postUploadDelayMs`, `noReset`, `saveOnSend`, `storageTarget`, `cliArgs`, `espruinoConfig`, `storagePreload`.
- **New:** `reset` mode, `chain`, `order`, `requirements`, `retryPolicy`, board JSON hints.
- **Sources:**
 - Manifest (`boards/<board>.json`): baud, default suites, future execution defaults.
 - Suite `testConfig.json`: per-suite overrides (timeouts, delays, default CLI args, chain defaults).
 - Test metadata header: fine-grained overrides per test.
 - CLI flags: `--pre-cli-delay`, `--post-cli-delay`, `--no-reset`, `--port`, `--board`, `--only`, `--show-config` (future), etc.

Fixtures & Data Assets

- **Examples:** Wi-Fi credentials, GPIO mappings, I2C addresses, Storage preload files.
- **Sources:**
 - Manifest fixture namespaces (e.g., `fixtures.wifi = "default"`).
 - Suite `testConfig.json`: required fixture blocks, default asset directories.

- Test metadata: `storagePreload` descriptors, fixture overrides.
- CLI `--fixtures`: selects the concrete JSON to inject.
- Test assets stored alongside suites (`tests/<suite>/assets/`).

Precedence Summary

Manifest defaults → Suite `testConfig.json` → Test metadata header → CLI flag/argument.

Next Actions

- Catalogue supported keys within the resolver and emit warnings for unknown entries.
- Seed `testConfig.json` files for active suites (Wi-Fi, runner metadata, basic async) to capture existing default behaviour.
- Ensure config snapshots are written to `runner-metadata` outputs for auditing.
- Document the schema so future suites can declare needs without runner code changes.